

Chio: A Peer-to-Peer Economy of Verifiable Work

The Chio Project

<https://github.com/bb-connor/arc>

October 2026

Abstract

An agent buying work from another organization needs more than a message channel and permission to pay. The provider needs a funded obligation; the buyer needs an agreed way to check the result; each organization needs to retain authority over its own resources. We describe a work contract joining these three conditions without a shared application operator. Each receiver authorizes execution locally. Signed evidence binds the agreed input, acceptance procedure, execution record and funding allocation. A separately funded subcontract can become payable even when its parent fails. We give a conditional composition argument, a reference implementation and a reproducible finite-state and contract-bytecode evaluation. An ordinary transactional escrow matches the monetary behavior on the same trace corpus: the contribution is the explicit composition and its implementation, not a new escrow primitive. The implemented profile trusts a selected verifier and settlement domain. It establishes a bounded basis for exchanging checked work; independent operation, lower integration cost and useful autonomous trade remain empirical questions.

1 Work between organizations

A buyer commissions a software review for 100 units. Its provider buys a specialist's analysis for 60. The specialist delivers acceptable work, but the provider crashes before completing the review. Two facts should survive: the buyer has not received its result, and the specialist has earned payment. Returning the buyer's funds must not erase the specialist's claim. Keeping the specialist paid must not require access to the provider's private scheduler, database or signing keys.

This example separates an agent workflow from a work economy. Within one application, a coordinator can schedule tasks, allocate a budget and interpret success. Between organizations, those decisions belong to different principals. A remote operator can fork its database, sign contradictory promises or stop responding. Its signature authenticates a statement; it cannot establish that all its promises are jointly funded. Likewise, a payment receipt proves a financial event, not that the agreed work was performed.

The unit of cooperation we propose is a *work contract*: immutable terms that identify a bounded task, its acceptance procedure and a funded obligation. The receiver admits the task under its own authority. Evidence of execution and acceptance can travel; the power to use the receiver's resources cannot be installed by the arriving evidence. Payment becomes collectible through the agreed settlement mechanism rather than a new discretionary promise from the buyer.

This construction uses existing cryptography and settlement. It does not create scarcity from signatures or equate useful work with consensus proof of work. Bitcoin resolves competing spends through a shared history under explicit assumptions [1]. Here the settlement domain supplies that

history; the harder application question is what evidence should make a particular work obligation payable, and how that event relates to local execution.

Our result is a concrete composition with a narrow demonstration. We identify the commitments each boundary must preserve, state the safety argument, and make its failure cases executable. We also implement the obvious alternative: an ordinary database escrow with the same transition rules. Its matching result is part of the evaluation. The protocol’s broader promise is that parties can implement one public contract and change partners without adopting a common application. That promise requires an independent interoperability and integration study; it does not follow from the safety argument.

2 Parties, authority and assumptions

A work edge connects a payer B and a provider P . A selected verifier V decides the agreed acceptance predicate, a custody service retains the relevant bytes, and a settlement domain R allocates and transfers funds. One organization may supply several services, but each role and key is explicit. Distinct keys alone do not establish independent administration. A provider on one edge can be the payer on another.

An agent chooses requests and may be malicious. The honest organization’s kernel mediates its resource access using locally activated trust roots, bounded capabilities and current policy. The remote operator may control its agents, signing keys, storage, messages and task implementation. The network may delay, duplicate, reorder or lose messages. Processes may fail between any two durable steps. Cryptographic authenticity cannot make a dishonest operator’s account of its own execution true.

The initial profile, F1, makes five assumptions. First, the honest party’s local enforcement and protected state work as specified. Second, signatures are unforgeable and the selected hashes resist collisions. Third, the settlement domain enforces exclusive allocations, atomic transitions and the chosen finality rule; its token transfers exactly the agreed units. Fourth, the pinned verifier applies the agreed predicate to the bound inputs and makes an eligible decision available within its window. Fifth, the parties can obtain the evidence and submit transactions needed to exercise a claim. Safety and liveness depend on different subsets of these assumptions.

The verifier is a trusted decision authority, even when its checker is publicly reproducible. Other readers can detect a wrong decision by rerunning the check; F1 does not make the settlement contract execute that check. A compromised verifier can approve bad work or withhold a decision. A compromised settlement domain can violate backing. These are central trust dependencies, not details of transport security.

Three ledgers describe different facts. The receiver’s authorization ledger answers whether an operation was admitted. The evidence record answers what was observed and checked. The settlement ledger answers which funds are reserved, payable or transferred. No record substitutes for the other two. In particular, a tool whose effect is unknown after a crash remains unknown even if the parties later release a payment hold.

We bound only the obligations admitted under the contract. The bound excludes unapproved off-protocol spending, opportunity cost, loss of disclosed data, verifier fees and transaction fees unless the terms explicitly reserve them. The reference experiment uses one fixed-price result and one subcontracting level. The composition argument applies to a finite set of distinct funded edges; it neither proves that a cyclic task graph finishes nor implements unrestricted recursive procurement.

3 The work contract

For exposition, write an agreement as

$$A = (B, P, I, C, D, F, T, L), \quad a = H(\text{canon}(A)).$$

I commits the input, C identifies the checker and acceptance policy, D defines permitted disclosure, F identifies the funding domain and terms, T sets the deadlines, and L names any parent agreement. Both parties sign the same canonical agreement. This tuple is a description of the common contract, not a replacement wire encoding. The artifact companion maps it to the versioned implementation objects.

An agreement has three bindings. *Authority binding* connects it to the receiver’s own admitted operation and bounded local grant. *Result binding* connects the exact input, output and acceptance procedure. *Funding binding* connects the agreement and result commitment to one allocation, beneficiary, asset and amount in one settlement domain. These bindings prevent a valid statement about one relationship from being reused as a statement about another. They do not establish useful work independently of the acceptance procedure.

3.1 Funding is an allocation

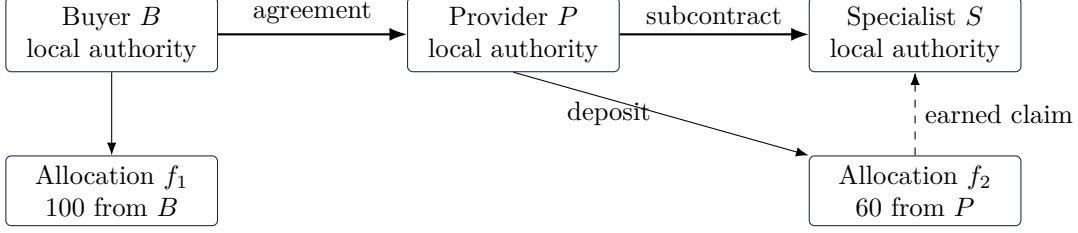
The payer deposits $q > 0$ units into an allocation f . The reference contract derives its allocation identifier from the chain, contract address and complete terms. The agreement digest, payer, beneficiary, verifier, asset, amount and four deadlines are immutable. A new payer agreement cannot be funded twice under the same payer and digest. The receiver checks the selected contract code and observation source before relying on an allocation.

A provider does not start merely because a payer supplies a signed balance. It requires an acceptable observation of the actual reserved allocation and a local admission decision bound to it. The binding does not atomically fuse a remote blockchain with a local database. Fresh observation, the deadline margin, retained operation identity and conservative recovery are necessary. If the chain reorganizes after an observation, the profile’s finality assumption has failed or the observation was premature. A local receipt cannot repair that.

3.2 Acceptance is an explicit decision

A submission commits the result and its evidence. A verifier checks the exact agreement, allocation, input, output, checker and custody record before issuing a decision. Missing evidence produces no decision. An available result that fails the predicate produces rejection. These outcomes must remain distinct: a verifier outage is not proof that the provider delivered bad work.

Off-chain artifacts use canonical JSON and Ed25519 signatures [2]. The reference escrow accepts a separate EIP-712 decision [3], binding the allocation, agreement, submission commitment, decision digest, acceptance bit, beneficiary, asset and amount. Its domain includes chain ID and contract address. The contract verifies the pinned signer’s decision, not the application predicate. The publication artifact specifies which digests cover bodies and which cover complete signed envelopes; those encodings cannot be interchanged.



Parent refund leaves f_2 intact.
Evidence crosses; each receiver admits locally.

Figure 1: Work dependencies and funding dependencies differ. Each obligation has its own backing; the figure depicts neither a shared application operator nor an absence of shared settlement and verification dependencies.

3.3 A child is a separate obligation

A parent link records provenance. It is not collateral and grants no authority. Before hiring a specialist, the intermediary must commit funds it already controls to a different allocation. In the opening example, the buyer deposits 100 and the intermediary deposits 60. The buyer's reserved 100 is not available for the intermediary to pledge again. If the intermediary cannot finance the 60, this profile cannot admit the subcontract. Credit or conditional netting would require a different contract and a different failure argument.

4 From agreement to payment

The financial state machine has seven states after funding:

$$\text{Funded} \rightarrow \text{Submitted} \rightarrow \begin{cases} \text{Payable} \rightarrow \text{Paid}, \\ \text{Rejected} \rightarrow \text{Refunded}. \end{cases}$$

A funded or submitted job without a decision can become **TimedOut** and then **Refunded**. There is no edge from **Payable** to timeout or refund.

Let $t_s < t_c < t_r < t_f$ denote submission, challenge, resolution and refund boundaries. The beneficiary submits by t_s . A first verifier decision is recorded only when $t_c < t \leq t_r$. Without a decision, refund is allowed only when $t > t_f$. The interval before decision is a timing boundary, not a claim that F1 implements a public challenge court. The exact verifier decision may be carried by any courier. Its first valid recording determines acceptance; an identical replay changes nothing, and a conflicting replay fails.

A claim is *earned* here when an accepting decision has been recorded in the settlement state under the selected finality rule. Merely producing an output or obtaining an off-chain signature does not earn this right. After that transition, the beneficiary may withdraw without a new payer signature. The reference contract imposes no later expiry on an earned withdrawal. Administrative pause and verifier-registry changes govern new funding and do not revise an existing claim. The recipient still needs transaction inclusion, fees and a functioning asset contract.

Local execution proceeds under its own durable operation identity. The kernel commits admission before dispatch, retains observed output and produces signed evidence. The funded integration carries the original allocation, operation, hold and authorization identifiers through verification

and recovery. Recovery reopens that history; it must not mint a fresh admission simply because a response was lost. A return whose occurrence cannot be established remains an unknown terminal. A later financial resolution has its own authority and does not rewrite the original observation.

For actual rail transactions, the research implementation records the exact signed transaction before broadcast. After a crash it reobserves or rebroadcasts those same bytes. A changed nonce, transaction or funding context is not an ordinary retry. This approach handles particular durable cutpoints; it is not an exactly-once guarantee for arbitrary tool effects.

4.1 Successful exchange and failed intermediary

In a successful exchange, the buyer and provider sign the terms, the buyer funds f_1 , and the provider admits one bound operation. The result and custody record identify the exact submitted bytes. The verifier reruns the selected check, records acceptance, and the provider withdraws 100. An offline reader can verify the signed evidence without possessing either party's private keys; current spendability still requires settlement state.

For the failed intermediary, the specialist's 60-unit child claim becomes **Payable** but remains uncollected. The parent produces no eligible submission and its 100 returns to the buyer after timeout. The specialist then withdraws the original 60. The provider bears the loss because it funded the child. This is neither a transfer of the buyer's risk to an unpaid specialist nor a claim that failed computation is free.

If instead the verifier never records a child decision, the child may time out and refund its payer despite the specialist having performed work. F1 protects recorded earned claims; it cannot guarantee timely adjudication against its own failed verifier. That distinction is the decisive difference between financial safety and an unconditional promise to pay for every useful result.

5 What composes

The following argument abstracts cryptography, finality and transaction execution into the assumptions of Section 2. It concerns admitted obligations, not the correctness of the entire implementation.

For each funding source u , let D_u be its fixed deposited endowment, A_u its unallocated balance, L_u its locked allocations, P_u its paid outflow and R_u its refunded outflow. Returned cash remains recorded as an outflow in this accounting interval; a later deposit opens a new interval. The conservation invariant is

$$D_u = A_u + L_u + P_u + R_u, \quad A_u, L_u, P_u, R_u \geq 0. \quad (1)$$

Proposition 1 (Bounded funded exposure). *If reservation moves $q \leq A_u$ from available to locked funds, and payment or refund moves an allocation's locked amount once to its corresponding outflow, every reachable state satisfies (1). Total paid and refunded outflow is at most the original deposit.*

Proof. The initial state is $(D_u, 0, 0, 0)$. Reservation subtracts and adds the same quantity within the sum. Payment and refund do the same across a different pair of accounts. Their guards preserve nonnegativity. Other transitions do not change money. Induction over the transition sequence proves the equality and bound. This argument requires the implementation to identify allocations uniquely and couple each state transition with its actual transfer. $\square$

Proposition 2 (Earned child independence). *Suppose each work edge has a distinct allocation with its own immutable terms, an earned claim can only remain payable or become paid, and a transition on one edge cannot modify another’s allocation. Then any sequence of parent failure, refund or completion transitions leaves an earned unpaid child collectible under its original terms.*

Proof. A transition concerning another edge leaves the child unchanged. A transition on the child preserves the set $\{\text{Payable}, \text{Paid}\}$. Induction therefore excludes child refund after acceptance. Distinct funding prevents a parent transfer from consuming the child’s reserve. Eventual collection further requires availability, an exercisable beneficiary key and transaction inclusion. $\square$

A small Lean specification checks conservation over arbitrary finite account traces, the paid-plus-refunded bound, persistence of the earned states, and noninterference of distinct map entries. A finite Python explorer checks concrete timing and role guards; selected traces are replayed against Solidity. These are complementary checks. There is no machine-checked refinement from the Rust kernel or Solidity contract to the Lean specification, and the arithmetic proof does not authenticate a signer or prove that a token is honest.

The capital cost follows directly. If E_{open} contains the distinct unsettled obligations, their reserve is $\sum_{e \in E_{\text{open}}} q_e$. A chain of promised future receipts cannot reduce this amount under F1. The example locks 160 units while the two obligations coexist, although the buyer’s task costs 100. Short milestones may reduce capital duration; they do not eliminate capital needs. Capital efficiency must be measured alongside success rate.

A payer-only signature cannot supply the missing exclusivity. Consider two receivers that see individually valid promises against the same deposit and have no common state or other non-equivocation mechanism. Each receiver’s view is indistinguishable from a run in which it is the only payee. If both accept on that evidence alone, the two promises can exceed the deposit. The argument identifies a missing shared fact; a conventional trusted ledger, a qualified rollback-resistant authority or a settlement chain can supply it. It is not an impossibility result about OAuth, A2A, capabilities or ordinary escrow.

6 Evidence and comparison

We evaluate the funded checkpoint and this paper’s artifact with a recorded source inventory. The companion distinguishes freshly executed checks from retained September process evidence and the October security integration. All reported executions use author-controlled infrastructure. Development-chain tokens have no economic value.

6.1 Transitions and negative controls

The finite graph starts with a three-unit parent and a two-unit child. It explores alternative submissions, conflicting and unauthorized decisions, payment, refund, expiry and execution uncertainty over the selected timestamps. Every reachable transition is checked against the invariants. Exploration is exhaustive only within this bounded alphabet and starting configuration. Funding and failed-transfer boundaries have separate unit tests.

The 118 shortest representative financial traces cover selected state-transition classes. The Solidity runner compares contract states, commitments, decisions and actual token balances with the model after each step. It does not replay all 132,192 graph transitions against bytecode. Additional

Experiment	Result	Boundary
Finite claim graph	5,508 states	Two prefunded edges, fixed symbolic actors and nine times-tamps
Transition checks	132,192	Conservation, terminal consistency, denial stability and uncertainty
Contract suite	143 passing	Compiled bytecode on a private development chain; includes 118 representative traces
Ordinary SQL comparator	312 matched steps	The same 118 traces; sequential trusted database
Upstream ERC-8183	76 passing	Unmodified upstream suite at the pinned commit
Paired escrow cases	20 passing	Same token, zero fees, capital and evaluator trust; local Cancun EVM
Lean specification	7 named results	Abstract transitions; no implementation refinement

Table 1: Fresh local evidence. Counts describe this artifact, not a production reliability estimate. Results and source hashes are checked by the build.

contract tests exercise failed and asymmetric transfers, unauthorized calls, duplicate funding, replay, verifier changes and delayed withdrawal. No confidence interval for deployed failure rates can be derived from this deterministic regression suite.

Two negative controls test whether the harness can detect the property it claims to enforce. A deliberately weakened model allows a payable job to expire; the explorer finds the sequence submission, acceptance, refund. A corresponding in-memory mutation of the compiled escrow is rejected by the earned-withdrawal regression. Neither negative control changes the source contract. Separately, the older escrow characterization exhibits a timely certificate losing payment eligibility at expiry. That historical contract has different semantics; the paper does not describe it as an attempted equivalent implementation of the new rule.

6.2 An ordinary transactional alternative

We implement the monetary state machine using ordinary SQL transactions and conditional updates, without Chio kernel code. It receives the same role identities, deadlines and predicate outcomes as the model. It checks every observable snapshot and source balance along the same representative traces. The result is 312 matched steps and no mismatch. Additional tests preserve an uncollected child’s payment after parent refund, deny reuse of its funds, and keep unknown execution distinct from verifier rejection.

This is evidence that the tested monetary safety is available through standard mechanisms. The comparator trusts the database operator; replacing it with the same external escrow gives an ordinary application access to the same scarce backing. We have not benchmarked that application’s full integration effort or implemented every native admission feature in it. Accordingly, the comparison supports neither a Chio speedup nor an impossibility claim against alternatives. Both implementations were written within this project, so it is also not an independently authored replication.

6.3 Comparison with upstream ERC-8183

We also test the existing agent-commerce implementation at upstream commit `142e669c1f`. Its 76 unchanged tests pass. We add 20 comparison tests, using the same mock token, zero fees,

a 100-unit buyer and a separately capitalized 60-unit intermediary. Each escrow uses its native authorization and deadline rules; distinct trusted evaluator addresses make decisions. The artifact retains compiled-bytecode traces, compiler and dependency pins, and reproduction after relocation.

Both implementations pay the child after parent refund, reject funding two 60-unit jobs from 100, and deny spending the unsettled parent reserve. The upstream implementation also supports a stronger milestone case: it pays an approved 60-unit claim from one 100-unit job, then refunds the remaining 40 when the job is rejected. Chio’s fixed-price allocation has no partial-settlement entrapment. These results reject an exclusive claim to either funded agent jobs or surviving child payments.

The observed differences concern state-machine policy. Chio separates recorded acceptance from withdrawal, rejects new acceptance after its resolution cutoff, and preserves earned withdrawal through a funding pause or allowlist change. Upstream completion pays immediately; after its expiry grace period, completion can still win if refund has not executed. Upstream refunds are blocked while paused. These are properties of the pinned implementations, not expressiveness limits of ERC-8183, whose evaluators and hooks admit extensions. No gas, performance or security superiority follows from the comparison.

6.4 Execution, verification and security integration

The existing funded prototype binds actual Rust-produced output to a separately implemented Python check, signed custody, a claim and payment. Its workload inventories supported OpenAPI authentication declarations. It checks the listed properties, not the security of a running API. September evidence records payment, rejection, missing custody, an earned child after parent refund, transaction recovery and native execution witnesses. Its later process experiments isolate role keys and pass public evidence between separate roles.

We reconciled funded source `7755d3762b` with security source `491f585e90` in integration commit `71e5cbc3bf`. Their changes overlap in durable admission, recovery and evidence handling; compatibility cannot be inferred from their independent tests. The integrated evidence snapshot passes 95 native tests, including all six opt-in chain tests, and 20 selected process scenarios covering authenticated transport, bilateral isolation, earned-child recovery, refund resolution and custody. The final source differs only by removal of a redundant dereference, checked by strict Clippy, four affected unit tests and a fresh authenticated payout/refund run. All roles remain under author control. Broader formal-source mapping, file-hygiene, hosted and deployment qualification remain open. The companion retains the exact integration record and each remaining gate.

The deterministic workload deliberately makes acceptance easy to inspect. It does not demonstrate an asymmetry between expensive production and cheap verification: a buyer can often compute this inventory itself. There is no held-out repair corpus, economic-margin result, live-agent advantage or independently operated company trial in the reported evaluation. Those omissions bound the broader economy claim rather than the tested escrow behavior.

7 Relation to existing mechanisms

Contract Net already organizes distributed work through announcements, bids and awards [4]. Agent Contracts unifies task specifications, resource constraints and success conditions, including delegated budget conservation [5]. Chio does not introduce contracting or hierarchical budget discipline. Our focus is the boundary between a remote operator’s local budget and an obligation backed by a

settlement authority the operator cannot rewrite.

AgentBound places deterministic governance before execution and binds decisions to verifiable receipts [6]. That overlaps substantially with local authorization and audit. A receipt about permitted behavior does not by itself reserve funds at a separate authority or decide application-specific output acceptance. Chio’s work contract explicitly joins those duties, while retaining their distinct assumptions.

A2A supplies tasks, messages, artifacts and discovery [7]; AP2 specifies signed commercial and payment mandates and their verification [8]. AP2 v0.2 leaves shopping-agent mandate delegation outside its scope and permits extensible authorization models. That is a scope choice, not evidence that its carriers cannot express this work contract. Likewise, OAuth token exchange leaves significant trust and token semantics to deployment profiles [9]. A receiver-issued credential and ordinary signed application fields can enforce receiver-owned authorization. Interoperability requires a defined profile and conformance tests, not a claim that other formats lack expressive power.

ERC-8183 is a particularly close construction: it specifies escrowed agent jobs, providers, evaluator-approved completion and expiry refunds [10]. The economic pattern is already present. The pinned upstream implementation additionally supports partial claims and relayable EIP-712 authorization; the specification and implementation differ in details. Section 6 tests their overlap with Chio directly. No new economic primitive follows from separating acceptance and withdrawal or from independently funding child jobs. ERC-8004 provides discovery, reputation and validation interfaces while leaving payments to other protocols [11]. A registry entry is evidence a receiver may consider, not a capability to use its resources. Both standards are drafts at the pinned snapshots.

Sagas explain how sequences of transactions use application-defined compensation [12]. A refund is such a financial action; it need not erase an already delivered effect. Our explicit unknown state prevents recovery from inventing a successful execution or a no-effect history. Durable workflows and transactional outboxes remain appropriate competing implementation techniques.

Contingent service payments and fair exchange have established constructions for linking payment to verification [13, 14]. F1 is weaker in an important respect: it relies on a selected verifier, rather than supplying a cryptographic proof of arbitrary useful work or complete fair exchange. Replacing that verifier with a proof system would require a restricted task language, cost measurements and a new analysis of availability and disclosure. The current implementation does not establish that extension.

The relevant comparison is thus a composition of competent implementations of these mechanisms. A missing field in a base specification is not an obstacle to that composition. Chio offers a particular contract and enforcement path; whether that common contract materially lowers the cost of changing partners remains the experiment that would distinguish protocol value from a large bespoke integration.

8 Where the argument stops

The two enduring dependencies in F1 are the acceptance authority and the settlement domain. Parties may choose them without choosing a shared task scheduler, but cannot dispense with their assumptions. An escrow administrator can restrict new participation; an asset issuer may freeze transfers; a verifier may censor or falsely approve. Immutable earned-state rules alone do not defeat those failures. No new token, global reputation system or universal identity registry is necessary for the demonstrated profile.

Verifiable work means work with an agreed check, not universally decidable quality. A test suite may miss a vulnerability. A proof may prove the wrong specification. A correct declared-authentication inventory may have little commercial value. The buyer must choose an acceptance procedure fit for its purpose and a disclosure policy it can tolerate. Once disclosed, data cannot be recalled by revoking a capability.

Evidence integrity is narrower than availability. A content hash detects substitution but cannot recover withheld bytes. A custody signature records a promise to retain data; it does not prove future service or erase storage costs. Likewise, a historical signed receipt can remain valid after authority expires, while fresh dispatch must still consult current authority and revocation state.

Our safety argument abstracts serial atomic rail transitions, and the local experiments use a private development chain or an owned chain fixture. They do not establish public-chain finality, independent observers, geographic failure tolerance, sustained denial-of-service resilience or hostile administration of the honest party's kernel. The Lean model and finite exploration have explicit boundaries; passing them does not prove the implementation as a whole.

The next decisive evaluation is an independently implemented provider and separately administered participants completing useful held-out work from the published contract. It must allow the alternative the same escrow, checker, models, budgets and ordinary application extensions. It should measure integration changes, failed-attempt cost, capital duration, verifier cost, recovery effort and buyer usefulness. Without those results, neither lower integration cost nor a profitable autonomous supply chain is an empirical conclusion of this paper.

9 Conclusion

A work relationship should bind permission to act, a rule for accepting the result and a funded obligation, while leaving each receiver in control of its resources. Chio makes those bindings explicit and preserves them through local execution, evidence exchange and financial recovery. Separately funded, recorded earned claims survive a parent's failure under the stated settlement and availability assumptions.

The mechanism has a concrete implementation and falsifiable boundaries. Its monetary rule is reproducible with ordinary escrow, and its strongest proposed benefit is still to be measured: making independently operated work relationships portable at lower integration cost. The protocol supplies a basis for that experiment and a precise account of what the experiment must preserve.

References

- [1] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. URL <https://bitcoin.org/bitcoin.pdf>.
- [2] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, 2020. URL <https://www.rfc-editor.org/rfc/rfc8785>.
- [3] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed Structured Data Hashing and Signing, 2017. URL <https://eips.ethereum.org/EIPS/eip-712>.
- [4] Reid G. Smith. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516.

- [5] Qing Ye and Jing Tan. Agent Contracts: A Formal Framework for Resource-Bounded Autonomous AI Systems. arXiv:2601.08815v3, 2026. URL <https://arxiv.org/html/2601.08815v3>.
- [6] Anuj Kaul, Qianlong Lan, and Pranay Gupta. Behavioral Governance for Autonomous AI Agents: The AgentBound Framework. arXiv:2606.30970v2, 2026. URL <https://arxiv.org/html/2606.30970v2>.
- [7] A2A Project. Agent2Agent Protocol Specification, 2026. URL <https://github.com/a2aproject/A2A/blob/65dadbd9b900fe2d970e05416eb432a91347cd2a/docs/specification.md>. Source pin 65dadbd9b900fe2d970e05416eb432a91347cd2a.
- [8] AP2 Contributors. Agentic Payment Protocol, version 0.2, 2026. URL <https://ap2-protocol.org/ap2/specification/>. Source pin and retrieval hash in the artifact’s sources.json.
- [9] Michael B. Jones, Anthony Nadalin, Brian Campbell, John Bradley, and Chuck Mortimore. OAuth 2.0 Token Exchange. RFC 8693, 2020. URL <https://www.rfc-editor.org/rfc/rfc8693>.
- [10] Davide Crapis, Bryan Lim, Tay Weixiong, and Chooi Zuhwa. ERC-8183: Agentic Commerce, 2026. URL <https://eips.ethereum.org/EIPS/eip-8183>. Draft, source snapshot 365b4c02879f3e882b91281d42b4f57b406205e9.
- [11] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. ERC-8004: Trustless Agents, 2025. URL <https://eips.ethereum.org/EIPS/eip-8004>. Draft, source snapshot 365b4c02879f3e882b91281d42b4f57b406205e9.
- [12] Hector Garcia-Molina and Kenneth Salem. Sagas. Technical Report CS-TR-070-87, Princeton University, 1987. URL <https://www.cs.princeton.edu/techreports/1987/070.pdf>.
- [13] Matteo Campanelli, Rosario Gennaro, Steven Goldfeder, and Luca Nizzardo. Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services. Cryptology ePrint Archive, 2017/566, 2017. URL <https://eprint.iacr.org/2017/566>.
- [14] Stefan Dziembowski, Lisa Ekey, and Sebastian Faust. FairSwap: How to Fairly Exchange Digital Goods. Cryptology ePrint Archive, 2018/740, 2018. URL <https://eprint.iacr.org/2018/740>.